

Apollo: Exploiting Spatial Multiplexing for Efficient Training of Large Multimodal Models

Anonymous Author(s)

Abstract

Training large multimodal models (LMMs) as a concurrent job batch in shared GPU clusters is increasingly common, yet remains inefficient because LMM pipelines exhibit strong module-level heterogeneity: different modality encoders and backbone stages impose mismatched compute footprints, creating persistent GPU bubbles under job-level isolation and coarse-grained scheduling. This paper presents *Apollo*, a multi-LMM training framework that exploits spatial multiplexing to colocate heterogeneous modules on the same GPU with explicit SM quota control, targeting the global makespan of a job batch. *Apollo* combines three key components. First, it uses a Lifecycle-Aware Green Context Controller to provide low-overhead isolation and fast context reuse for fine-grained sharing. Second, it introduces a Bottleneck-Aware performance model that predicts colocation interference by modeling contention features, enabling accurate plan evaluation. Third, it employs HASP, a heterogeneity-aware sub-partition scheduling algorithm that searches efficient quota-aware colocation plans under tight online budgets. We evaluate *Apollo* on representative LMM workloads (ImageBind, OFASys, Qwen3-VL, and Unified-IO 2) on 16- and 32-GPU clusters and via trace-driven simulation at larger scales. On a 32-GPU cluster, *Apollo* achieves a $1.14\times$ – $1.33\times$ makespan speedup over the state-of-the-art scheduler Spindle, with larger gains of up to $1.49\times$ over DistMM and $1.71\times$ over NVIDIA MPS under saturated settings. At scale, *Apollo*’s advantage persists in simulation, reaching up to $1.25\times$ speedup over Spindle and $1.51\times$ over DistMM at 1024 GPUs.

1 Introduction

Large multimodal models (LMMs) [4, 5, 11, 15, 28, 50, 56] couple a Transformer [58] backbone with modality-specific encoders and fusion components, forming a heterogeneous execution pipeline [23, 33] where different modules exhibit sharply different compute intensity and scaling behavior. In modern development workflows [18, 46?], LMM training rarely occurs as a single isolated job: practitioners increasingly train multiple related LMM jobs collaboratively and evaluate them as a job batch whose end-to-end turnaround is dictated by the batch makespan.

Consequently, when orchestrating concurrent LMM training jobs on a shared GPU cluster, the key objective is to exploit each job’s heterogeneity [65] and choose an optimal orchestration strategy that minimizes the overall makespan of the job batch.

However, existing training orchestrators [13, 21, 65] largely follow a two-step decomposition: they first perform *inter-job scheduling* to determine a job execution order, and then conduct *intra-job mapping* to place a job’s modules onto GPUs

This job-granular orchestration model [21, 63] fundamentally limits fine-grained temporal alignment across jobs: once jobs are isolated at decision time, idle gaps [21] created by one job’s low-intensity modules cannot be filled by modules from other jobs, leaving substantial GPU idle time [44] during the training process.

Worse, due to module heterogeneity inside LMMs [21, 60, 65], a single module often cannot saturate the GPU; thus, there remains significant space to improve cluster throughput and accelerate training completion if we can mix complementary modules more finely than the job boundary permits.

Our insight is to introduce a *spatial multiplexing dimension* into LMM orchestration. Instead of treating a GPU as an indivisible resource [13, 21, 60, 65] owned by one job at a time, we allocate explicit GPU compute quotas [41] to individual modules based on their heterogeneity characteristics, enabling multiple modules to execute concurrently on the same GPU. By matching complementary modules through quota-based sub-GPU partitioning, spatial multiplexing converts otherwise wasted cycles into effective throughput and shortens the overall makespan of the multi-LMM training job batch.

Realizing efficient spatial-multiplexed orchestration is challenging. First, the system requires a practical stack to enforce fine-grained compute partitioning [37, 38, 40] on GPUs, so that quota assignments are both precise and low-overhead. Second, to evaluate candidate colocation plans, we must accurately predict module iteration time under specific compute quotas, including the additional interference overhead [25, 59] induced by colocation [12]. Third, the scheduler must handle a large hybrid decision space [63]—discrete choices of module grouping and ordering, plus continuous choices of quota splits—while still finding an optimal plan within a bounded scheduling time.

In this paper, we propose *Apollo*, a spatial-multiplexed parallel training system for multi-LMM job workloads. *Apollo* consists of three components: *ApolloPredictor* profiles jobs and characterizes heterogeneity; *ApolloScheduler* searches for a makespan-optimal orchestration plan with quota-based spatial multiplexing; and *ApolloEngine* instantiates and executes the plan efficiently at runtime.

ApolloPredictor performs fine-grained profiling for each submitted job using NVIDIA MPS [40] to expose module-level resource demands. From these profiles, *ApolloPredictor* derives each model’s scaling curve as a function of GPU count and SM compute quota, which serves as the key input to *ApolloScheduler*’s search.

ApolloScheduler jointly decides cross-job module colocation and quota assignment. To make plan evaluation accurate under spatial multiplexing, *ApolloScheduler* models colocation interference by capturing bandwidth-induced slowdowns [12], and introduces a bottleneck-aware colocated overhead model based on DRAM throughput to predict the extra latency beyond quota-only scaling. To balance optimality and search efficiency in the hybrid decision space, *ApolloScheduler* further proposes Heterogeneous-Aware Sub-Partition Scheduling (HASP), a heterogeneity-driven search strategy that aggressively prunes unpromising merges while preserving a monotonic improvement property, enabling efficient discovery of high-quality orchestration plans.

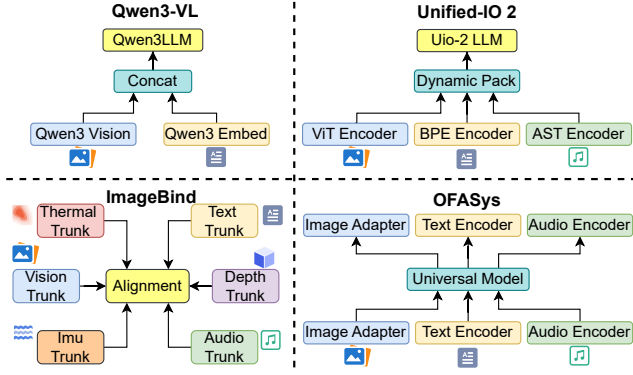


Figure 1: Representative LMM module graphs used in this work: Qwen3-VL, Unified-IO 2, ImageBind, and OFASys. Each model is decomposed into modality-specific encoders/adapters and a shared fusion/universal component.

ApolloEngine enforces the orchestration plan and runs the actual training processes. On each GPU, *ApolloEngine* instantiates a Green Context Controller to manage quota-specific CUDA Green Contexts [38] across the full lifecycle, ensuring that quota switching remains lightweight and does not accumulate on the critical path. Meanwhile, *ApolloEngine* continuously collects per-iteration runtime statistics during training and feeds them back to refine each job’s scaling curve, improving prediction fidelity and further stabilizing scheduling decisions.

We evaluate *Apollo* with physical experiments on an NVIDIA A800 GPU cluster. Across diverse multi-LMM job workloads, *Apollo* reduces end-to-end makespan by 14%–33% over state-of-the-art baselines, demonstrating that spatial multiplexing with low-overhead control and efficient, accurate scheduling is practical for LMM training. We further conduct profile-driven simulations at larger cluster scales (256, 512, and 1024 GPUs), where *Apollo* achieves up to 24.17% additional makespan reduction.

2 Background

2.1 LMM Module Heterogeneity

LMM adopt a modular pipeline that combines modality encoders with a shared backbone to support cross-modal interaction and generation or alignment. Figure 1 shows representative designs, including Qwen3-VL [8], Unified-IO 2 [30], ImageBind [16], and OFASys [7]. This modularization implies that a single training iteration executes a sequence of heterogeneous modules rather than a uniform compute block.

Different LMMs instantiate different component sets, and their training proceeds as a sequence of pipeline stages whose executed modules depend on the model architecture and the input modality mix. Table 1 indicates that the compute demand can vary across stages within a single iteration. For example, Qwen3-VL-8B exhibits a pronounced gap between a backbone stage estimated at 22.27 TFLOPs per 1K tokens and a vision stage at 2.58 TFLOPs, while Unified-IO 2 includes a T5 stage estimated at 16.70 TFLOPs alongside modality stages around 1.48 TFLOPs.

Table 1: Module-level configurations and per-iteration compute estimates for the representative LMM workloads.

Model	Module	Layers	Dim.	TFLOPs
Qwen3-VL-8B	Qwen3LLM	36	4096	22.27
	Qwen3 Vision	27	4096	2.58
	Qwen3 Embed	1	4096	0.15
	Concat	1	4096	0.02
Unified-IO 2	UIO-2 LLM	48	3072	16.70
	ViT Encoder	11	768	1.48
	AST Encoder	11	768	1.06
	BPE Encoder	1	3072	0.10
	Dynamic Pack	2	3072	0.08
ImageBind	Vision Trunk	24	1024	4.17
	Audio Trunk	12	768	2.09
	Text Trunk	12	768	1.04
OFASys	Universal Model	36	1280	4.80
	Image Adapter	8	1280	1.35
	Text Adapter	4	1280	0.72
	Audio Adapter	6	1280	0.95

Moreover, the same imbalance also appears at the lightweight-module level: OFASys similarly mixes a 4.80-TFLOPs universal model with adapters ranging from 0.72 to 1.35 TFLOPs, and ImageBind’s modality trunks span from 1.04 to 4.17 TFLOPs.

Overall, a training iteration alternates between pipeline stages with substantially different compute demands, creating intrinsic heterogeneity across both backbone and modality-specific components.

2.2 Concurrently Train multi-LMM job in Shared Clusters

Industrial LMM development commonly proceeds as a related job set that must advance together on a shared GPU cluster. Kubeflow Pipelines [24] and Apache Airflow [6] capture this practice by representing executions as dependency graphs, where ultimate artifacts become available only after all required jobs finish. Recent industry products illustrate that deployments often require a bundle of related LMMs to complete together. Google presents Gemini [14, 17] with multiple sizes, Ultra, Pro, and Nano. These variants are part of a unified engineering effort and training stack. ByteDance’s Doubao [9] is a coordinated release of multiple models within one product line [1], and Meituan’s LongCat-Image includes multiple model variants supporting both generation and editing [2, 3].

Under this workflow, the primary performance objective becomes the *makespan* of the related job set, because the resulting release is useful only after all dependent training and validation jobs complete. This emphasis aligns with cluster scale economics in multi-LMM job settings, where operators track tail completion behavior and utilization to sustain iteration speed under contention.

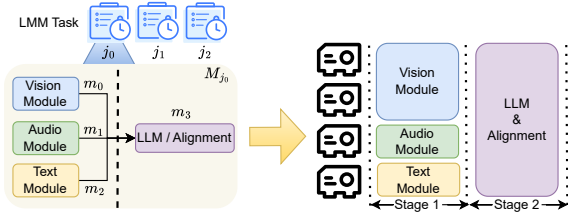


Figure 2: Problem construction of multi-task LMM training.

2.3 The Goal of Our Work.

We aim to improve cluster throughput for multi-LMM job training by reducing the end-to-end completion time of a concurrently running job set, since the overall completion time of the job set governs the iteration speed of a development pipeline. Our key measure is how the scheduler maps each training module to an appropriate set of GPUs and assigns a corresponding compute allocation during training, which directly determines the attainable throughput under shared-cluster execution.

3 Motivation

In multi-LMM job settings, training pipelines exhibit pronounced heterogeneity that makes structural inefficiency hard to remove with conventional job-level scheduling policies [18, 46]. Table 1 illustrates this heterogeneity: modality encoders vary widely in architectural configuration, and different jobs may compose different encoder stacks and fusion workloads [21, 65]. As a result, the computation demand and scaling behavior can differ substantially across jobs, creating persistent imbalance that job-level allocation alone cannot reconcile.

These results indicate that substantial GPU capacity remains underutilized even under continuous multi-job LMM training demand.

3.1 Problem Construction

Inter-job orchestration. As illustrated in Figure 2, we model concurrent multi-task LMM training as stage-wise execution across jobs. Let $\mathcal{J}$ be concurrent jobs and $s \in \{1, \dots, S\}$ be pipeline stages. For each stage s , we denote all executable modules from all jobs as $\mathcal{U}_s$. Inter-job orchestration decides a stage-level execution order

$$\Pi_s = \langle B_{s,1}, \dots, B_{s,L_s} \rangle,$$

where each $B_{s,\ell} \subseteq \mathcal{U}_s$ is one execution group, and Π_s is an ordered partition of $\mathcal{U}_s$.

Intra-job mapping. For each group $B_{s,\ell}$, intra-job mapping assigns modules to GPUs and sets compute quotas. Let $\mathcal{G}$ be the GPU set, $\phi_{s,\ell}(m) \in \mathcal{G}$ be the GPU of module m , and $\rho_{s,\ell}(m) \in (0, 1]$ be its quota. The only required capacity constraint is:

$$\sum_{m \in B_{s,\ell}: \phi_{s,\ell}(m)=g} \rho_{s,\ell}(m) \leq 1, \quad \forall s, \ell, \forall g \in \mathcal{G}. \quad (1)$$

The unified formulation for multi-LMM job orchestration. Given orchestration order Π_s and mapping/quota (ϕ, ρ) , stage s

has duration $T_s(\Pi_s, \phi, \rho)$, where $T_s(\cdot)$ is provided by the runtime performance model.

Thus, the optimization is:

$$\begin{aligned} \min_{\{\Pi_s\}, \phi, \rho} \quad & \sum_{s=1}^S T_s(\Pi_s, \phi, \rho) \\ \text{s.t.} \quad & \sum_{m \in B_{s,\ell}: \phi_{s,\ell}(m)=g} \rho_{s,\ell}(m) \leq 1, \quad \forall s, \ell, g, \\ & \phi_{s,\ell}(m) \in \mathcal{G}, \quad 0 < \rho_{s,\ell}(m) \leq 1, \quad \forall s, \ell, m. \end{aligned} \quad (2)$$

3.2 The Limitations of Existing Solutions

Most existing approaches improve concurrent LMM training by optimizing scheduling decisions within a limited scope [54, 67]. By design, they commit early to specific structural assumptions on how a training job executes and how resources are assigned, which shrinks the feasible plan space and leaves avoidable fragmentation when multi-LMM training jobs run together. We group prior systems into two families: *job-isolated schedulers* and *job-coordinated schedulers*. We discuss the constraints and limitations of each.

3.2.1 Job-isolated schedulers.

A common design point in prior work is to optimize each LMM training job in isolation, without enabling fine-grained scheduling across jobs. Representative systems include Optimus [13], Dist-Train [65], and DistMM [21]. Under our unified formulation, these designs constrain inter-job orchestration to job granularity: each execution group in a stage can contain modules from only one job.

Formally, for every stage s and group $B_{s,\ell}$:

$$|\{j \in \mathcal{J} : B_{s,\ell} \cap \mathcal{M}_{j,s} \neq \emptyset\}| = 1. \quad (3)$$

Eq. (3) is the job-isolation constraint in the same notation as Eq. (2): it forbids cross-job module interleaving inside any stage-level execution group.

For a strict order $j_0 < j_1$, job j_1 can start only after j_0 finishes:

$$T_{j_1}^{\text{start}} \geq T_{j_0}^{\text{end}}. \quad (4)$$

This design simplifies enforcement and preserves scalability [19, 31, 67], but it commits to a constrained plan space. When one job exhibits phase-level imbalance, its slack cannot be backfilled by modules from other jobs, which increases waiting at synchronization boundaries and hurts overall makespan.

Limitations of job-isolated scheduling. We quantify this inefficiency on a single node with 8xH100 GPUs by running two concurrent LMM training jobs that follow an OFASys-style model structure. As shown in Figure 3, heterogeneous modality modules exhibit highly imbalanced compute footprints. Across the run, these ‘gpu idle bubbles’ account for 17.1% of the total GPU card-time, yet job-isolated scheduling cannot reclaim them: idle ranks cannot be backfilled by modules from the other job, and the two jobs must wait at synchronization boundaries and proceed in a job-by-job order. Therefore, the achievable utilization improvement is fundamentally bounded even with sophisticated intra-job orchestration.

*We refer to GPU idle bubbles as periods where a GPU is allocated to a training job but remains idle or underutilized due to pipeline imbalance and synchronization.

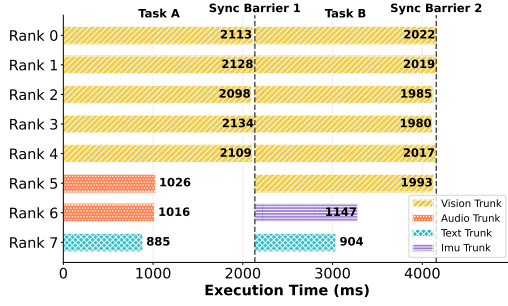


Figure 3: Job-isolated Module Orchestration. The orchestrator cannot align module execution in time across modules within a LMM training job.

3.2.2 Job-coordinated schedulers.

Job-coordinated schedulers [10, 62] aim to orchestrate multiple concurrent LMM training jobs under a single coordinated plan. They explicitly schedule modules across different jobs [10], while ensuring that the execution order within each job respects its LMM DAG dependencies [62][†]. This job-coordinated design expands the temporal search space [55] beyond per-job isolation, since the scheduler can decide which job’s ready modules should advance at a given time, subject to each job’s DAG constraints.

Spindle [60] follows this design and constructs a staged execution plan that coordinates modules from multiple jobs around shared stage boundaries. To pursue tighter stage alignment under these constraints, Spindle further splits modules into finer-grained layer groups, so that the stage completion times of different nodules match more closely.

Limitations of job-coordinated scheduling. Per-rank GPU utilization under job-coordinated scheduling is sampled at 100 ms granularity, which exposes how well stage boundaries align across ranks. With tight alignment, utilization drops and recoveries would occur nearly simultaneously at stage transitions. Instead, Figure 4 shows repeated misaligned collapses, where some ranks enter idle periods while others remain active, indicating that stage completion is still dominated by stragglers and temporal alignment is not achieved.

Moreover, the stage construction in job-coordinated scheduling introduces additional synchronization points [20, 32, 68]. Non-compute effects such as collective communication, bandwidth pressure, and input-pipeline stalls contribute fixed overheads and variance [57, 64]. As the number of stages increases, these overheads accumulate and amplify tail periods, which is consistent with the frequent utilization valleys observed in Figure 4. In our profiling run, the resulting GPU idle time accounts for 12.17% of the total GPU time.

3.3 Insight and Opportunity

The unified formulation reveals that inefficiency in multi-LMM jobs training is not primarily caused by insufficient GPUs, but by a missing capability in how shared clusters are orchestrated. Existing

[†]DAG dependencies capture the precedence constraints among modules in the LMM structure.

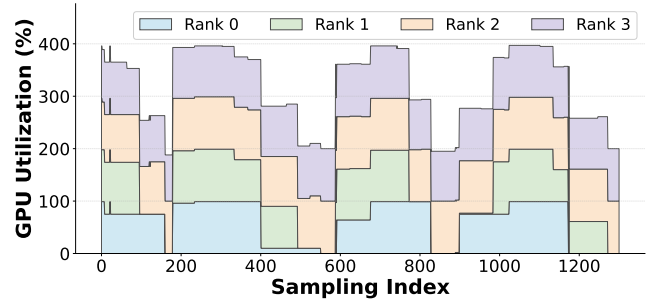


Figure 4: Job-coordinated GPU utilization under Spindle. Per-rank GPU utilization profile during concurrent multi-LMM jobs training.

schedulers primarily optimize along the temporal dimension by deciding which jobs or modules run next, while treating GPUs as non-shareable [19] across jobs. As a result, GPU idle gaps persist under concurrent workloads because unused capacity on one job’s GPUs cannot be reclaimed [65] by modules from other jobs. This observation points to an opportunity orthogonal to temporal re-ordering, namely exploiting spatial sharing [36] on GPUs to convert idle cycles into effective progress.

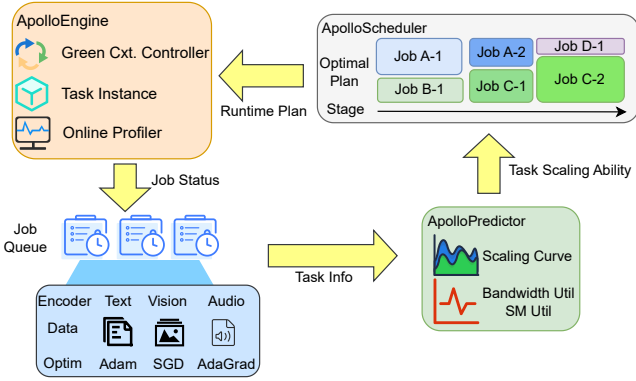
Key insight. Inter-job module mixing should be realized through spatial GPU sharing rather than only temporal coordination. Unlike job-coordinated schedulers such as Spindle that seek stage alignment by interleaving execution in time under GPU exclusivity, we focus on reclaiming fine-grained slack by allowing modules from different jobs to share the same GPU.

At a given scheduling step, the system can place modules from multiple jobs on the same GPU and assign each module a controlled compute share, so that idle capacity is translated into useful training progress. This GPU-level spatial mixing expands the deployment space beyond job-level decisions and enables better alignment of heterogeneous pipeline phases across jobs, improving the makespan of the concurrent job set.

Enabling opportunity. Spatial inter-job module mixing is practical because modern GPUs already support safe spatial sharing in technique. At the hardware level, MIG partitions a GPU into isolated instances with dedicated resources [37]. At the runtime/driver level, MPS [40] enables concurrent execution from multiple CUDA contexts on the same GPU. CUDA Green Contexts [38] further expose finer-grained resource partitioning by allocating disjoint subsets of SMs to different contexts.

These mechanisms provide the technical foundation for assigning controlled GPU shares to modules from different jobs and reclaiming otherwise idle capacity.

Our direction. We pursue a module-centric orchestration strategy that jointly schedules modules across concurrent LMM jobs and leverages spatial GPU sharing as the key enabling mechanism. This approach targets the missing capability in prior work and expands the feasible plan space beyond job-isolated scheduling and GPU-exclusive job coordination, providing a clear path toward higher cluster throughput under concurrent multi-LMM jobs training.


 Figure 5: The overview of *Apollo*.

4 Solution

Apollo addresses low cluster utilization in multi-LMM jobs training by enabling module-level spatial colocation with sub-GPU compute partitioning.

4.1 Overview

As shown in Figure 5, *Apollo* consists of three core modules: *ApolloPredictor*, *ApolloScheduler*, and *ApolloEngine*.

ApolloPredictor profiles each training job to extract its module DAG and the performance metadata needed for scheduling. These profiles serve as the source data for evaluating candidate colocations and compute splits.

ApolloScheduler takes the cluster resource state together with the profiling metadata and searches an orchestration plan for the job batch. It designs a batch-level scheduling strategy that targets low makespan while respecting per-job constraints.

ApolloEngine executes the plan at runtime of each job instance. It initializes *Green Context Manager* that performs full lifecycle management of Green-Context-enabled streams [45]. *Online Profiler* records module runtimes, and continuously refines the fitted SM performance model to improve plan quality over time.

4.1.1 Expanding the Unified Plan Space with SM Quotas

We extend the unified formulation from whole-GPU mapping to sub-GPU spatial partitioning. The key idea is to allocate a compute quota on a GPU to each executing module within a wave, rather than granting exclusive GPU ownership. This refinement enables module-level colocation across jobs and turns previously idle gaps into usable compute capacity.

4.1.2 Heterogeneous Quota-to-latency Scaling.

A quota-based plan requires an explicit mapping from SM quota to module runtime [27]. We provide this through an SM performance model. For each module type m , we profile isolated per-iteration runtime under different SM quotas and GPU degrees, and fit a scaling surface:

$$T_{\text{solo}}(m; d, s) \approx f_m(d, s). \quad (5)$$

where d is the GPU degree and s is the SM quota.

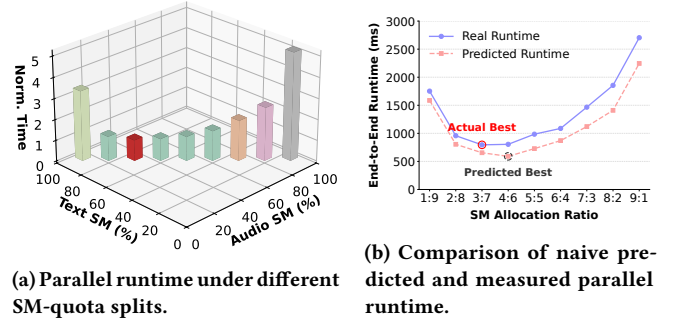


Figure 6: SM-quota impact and prediction discrepancy in two-module co-execution.

This interface captures two facts that matter for scheduling. Runtime is not proportional to quota because speedup saturates, and scaling behavior differs across modules.

4.1.3 Challenges

Efficient realization of sub-GPU partitioning. The core difficulty is to enforce module-granular compute quotas at iteration timescales with minimal control-plane overhead. This is hard because existing mechanisms provide partitioning at mismatched granularity. MIG operates at the GPU-instance level, and MPS operates at the process level, while modules of a training job typically execute within one process on a GPU [48]. Time-sliced sharing [39] further lacks enforceable quotas [42] and introduces high variance. As a result, realizing lightweight, fine-grained compute partitioning remains challenging in practice.

Accurate prediction of colocation runtime. Sub-GPU partitioning makes colocation quota-sensitive: for the same collocated set, different SM splits can yield very different makespans. Figure 6(a) shows that shifting the quota changes the bottleneck module and can significantly change the parallel time. A common baseline is bottleneck evaluation,

$$T_{\text{coloc}}(C) \approx \max_{m \in C} T(m; x). \quad (6)$$

Existing approaches often approximate $T(m; x)$ using isolated scaling only. This is unreliable because colocation introduces interference and quota-dependent loss, causing systematic deviation from the naive estimate. Figure 6(b) illustrates the consequence: the predicted curve has large error across SM ratios and selects a suboptimal split, thereby missing the best colocation plan.

Efficient search for multi-LMM jobs scheduling. Compute-quota colocation yields a hybrid discrete-continuous search space. Even within one wave, deciding colocation groups is combinatorial: for n frontier modules, the number of set partitions is the Bell number B_n . For each candidate group, the scheduler must further solve a continuous quota assignment under SM and memory budgets. Naive enumeration is therefore super-exponential in n and infeasible online.

Coarse pruning can discard high-gain colocations and lead to large optimality gaps, which then amplifies tail completion. This

motivates a solver that reduces the effective search complexity while preserving high-quality solutions.

Table 2: Python GC Mechanism Overhead

SM Allocation	SM Count	Overhead (ms)
10%	16/132	27.69
30%	40/132	28.24
50%	64/132	27.76
70%	96/132	29.18
90%	114/132	27.54

4.2 Mechanism: Green Context Controller

Mechanism Requirement. The mechanism must support sub-GPU compute control, remain lightweight on the iteration critical path, and require minimal changes to existing training stacks.

MIG partitions a GPU at the instance level, which is too coarse to regulate compute among modules inside a single training job. MPS partitions compute at the process level, but typical training implementations place all modules of a job on the same GPU within a single process. Forcing modules into separate processes to leverage MPS introduces GPU time-slicing effects and cross-process coordination overhead, which can significantly reduce training efficiency. A third common approach is time-sliced GPU sharing, such as scheduler-level GPU time-sharing or priority-based multiplexing among CUDA streams [45]. These approaches provide weak isolation and do not offer explicit compute quotas.

Green Context [35] is an NVIDIA primitive that enforces an SM quota on a CUDA stream, enabling fine-grained per-stream compute allocation. CUDA 13 [43] further improves Green Context support and reduces enforcement overhead.

Green Context Stream Lifecycle Management. Green-Context-enabled streams must be created with a specific SM quota, and changing the quota requires switching to a different stream instance. In iterative LMM training, modules are launched repeatedly at high frequency, so creating and destroying quota-specific streams on demand places control-plane work on the critical path.

Python-based training stacks further amplify this cost. Stream objects and their associated Green Context states are reclaimed by Python garbage collection, which introduces non-deterministic churn [47] and delays reclamation until GC is triggered. As a result, repeated create-destroy cycles accumulate overhead and degrade effective stream utilization.

Table 2 quantifies this effect. Across representative SM quotas, a single create-destroy cycle costs 27.54–29.18 ms. This overhead is largely insensitive to the quota value, meaning it cannot be avoided by selecting a particular split. When paid repeatedly across iterations and across GPUs, the cost becomes comparable to the execution time of many fine-grained modules, turning control-plane management into a throughput bottleneck.

These results motivate a pooled lifecycle design. By pre-creating a small set of quota-specific streams once and reusing them across iterations, runtime quota changes reduce to selecting an existing stream rather than constructing a new one. This avoids GC-driven

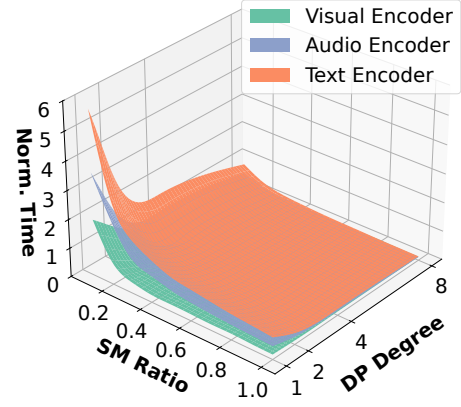


Figure 7: Heterogeneity of module scaling curves in the performance model.

churn, removes creation overhead from the iteration critical path, and makes sub-GPU compute partitioning deterministic and low overhead.

4.3 Performance Modeling

Building on Section 3.1, we model latency under quota-based spatial sharing. For stage plan $\Pi_s = \langle B_{s,1}, \dots, B_{s,L_s} \rangle$, intra-stage mapping is defined by $\phi_{s,\ell}(m) \in \mathcal{G}$ (GPU assignment) and $\rho_{s,\ell}(m) \in (0, 1]$ (SM quota) for each module $m \in B_{s,\ell}$.

To enable colocation, each GPU must satisfy:

$$\sum_{m \in B_{s,\ell}: \phi_{s,\ell}(m)=g} \rho_{s,\ell}(m) \leq 1, \quad \forall s, \forall \ell \in \{1, \dots, L_s\}, \forall g \in \mathcal{G}. \quad (7)$$

Given $(B_{s,\ell}, \phi_{s,\ell}, \rho_{s,\ell})$, group runtime is predicted by

$$\tau_{s,\ell} = T(B_{s,\ell}, \phi_{s,\ell}, \rho_{s,\ell}), \quad (8)$$

where $T(\cdot)$ is learned from profiling data (Figure 7). Stage latency is

$$\Delta_s = \sum_{\ell=1}^{L_s} \tau_{s,\ell}, \quad (9)$$

and is used by Eq. (2) to evaluate candidate schedules.

Module SM performance scaling modeling. The scheduler requires a constant-time interface to predict how a module’s runtime changes with GPU degree and compute quota. For each module type m , we profile isolated per-iteration runtime under different GPU degrees and SM quotas, and fit a smooth scaling surface as Eq. (5)

Bottleneck-aware Colocated Overhead modeling. Sub-GPU partitioning controls compute by assigning SM quotas, but it does not isolate the memory hierarchy. When multiple modules run on the same GPU, they still compete for shared DRAM bandwidth. [12] As the colocated slowdowns track the saturation of memory throughput rather than the nominal SM quota.

Figure 8 shows that a module’s standalone DRAM bandwidth throughput is strongly quota-dependent and differs across module types. As SM quota increases, bandwidth demand rises and approach a saturation region. Therefore the colocation overhead

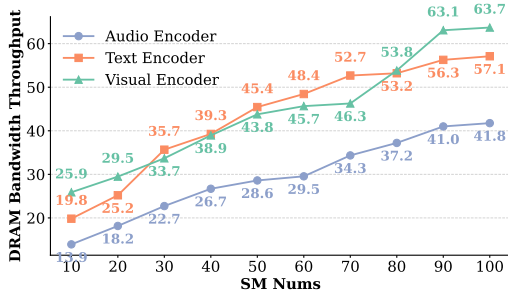


Figure 8: Standalone DRAM bandwidth demand varies with SM allocation and correlates with interference.

should be modeled as a function of quota-dependent bandwidth demand.

For each module type m , we profile its standalone DRAM throughput at several quota points and fit a monotone, shape-preserving piecewise-cubic interpolant to capture non-linear and saturating trends:

$$\widehat{B}(m; s) = g_m(s). \quad (10)$$

This provides $O(1)$ query cost while avoiding the artifacts of piecewise-linear fitting.

Multi-module contention features. Given a colocated group C sharing one GPU with quotas $\{s(m)\}_{m \in C}$, define each module’s predicted bandwidth demand

$$b_m = g_m(s(m)), \quad \forall m \in C. \quad (11)$$

We derive two statistics that scale to arbitrary $|C|$ and summarize both aggregate pressure and skew:

$$X_1(C) = \sum_{m \in C} \frac{b_m}{\kappa}, \quad X_2(C) = \sum_{m \in C} \left(\frac{b_m}{\kappa} \right)^2 \quad (12)$$

where κ is a normalization constant. To capture multi-module interaction intensity, we use the cross-term implied by the second moment:

$$X_\times(C) = X_1^2(C) - X_2(C) = 2 \sum_{m < m'} \frac{b_m}{\kappa} \frac{b_{m'}}{\kappa}. \quad (13)$$

X_1 reflects total bandwidth pressure, while X_2 and $X_\times$ capture how pressure concentrates and compounds across multiple modules, which is critical when $|C| > 2$.

We predict the colocation-induced inflation with a constant-time regressor over these features:

$$\Delta_{\text{overhead}}(C) = w_0 + w_1 X_1(C) + w_2 X_2(C) + w_3 X_\times(C). \quad (14)$$

This form is explicitly multi-module: it generalizes beyond pairwise colocation without changing the feature definition or inference cost.

Finally, the colocated runtime estimate remains:

$$T_{\text{coloc}}(C) = T_{\text{bn}}(C) + \Delta_{\text{overhead}}(C), \quad (15)$$

where $T_{\text{bn}}(C) = \max_{m \in C} f_m(d(m), s(m))$ is the bottleneck time under the quota-to-latency scaling model.

4.4 Heterogeneous-Aware Sub-Partition Scheduling

With module-level colocation and sub-GPU quotas, scheduling requires deciding how to partition n ready modules into colocated groups and which (d, s) configuration to assign under GPU capacity constraints. The grouping space alone is the Bell number B_n , and each group evaluation requires a feasibility-and-cost check. We therefore use a fast greedy scheduler that reduces decision latency while retaining high makespan gains.

4.4.1 Profile-Driven Synergistic Module Consolidation

Stage-based decision unit. We schedule at the granularity of a stage and solve one stage at a time. Each stage exposes a set of ready modules that can execute concurrently subject to resource budgets. The batch makespan is the sum of predicted stage times, consistent with the stage-synchronous execution model in Section 3.1.

Bottom-up merging with conservative improvement. For a stage module set $\mathcal{M}$, the solver starts from a serial baseline $\mathcal{P} \leftarrow \{\{m\} \mid m \in \mathcal{M}\}$. It then iteratively merges two groups only when the merge strictly reduces predicted stage time. For groups $C_a, C_b \in \mathcal{P}$, define the merge saving:

$$\Delta(C_a, C_b) = (T(C_a) + T(C_b)) - T(C_a \cup C_b), \quad (16)$$

where $T(C)$ is the predicted runtime of colocated group C computed from profiled scaling and overhead models (Section 4.3).

At each step, we apply the merge with the largest positive Δ ; if no merge yields $\Delta > \epsilon$, we stop. This rule avoids unstable early decisions and provides a monotone improvement path from a feasible baseline.

Profile-guided stage evaluation. The key subroutine is a stage evaluator that returns the minimal feasible runtime for a group under the profiled performance model:

$$T(C) = \min\{T \in \mathcal{T}(C) \mid \text{FEASIBLE}(C, T)\}. \quad (17)$$

Here $\mathcal{T}(C)$ is a discrete candidate set derived from profiled configuration runtimes, and FEASIBLE checks whether modules in C can be placed on the available GPUs with quotas under SM and memory budgets. We memoize $T(C)$ for previously seen groups to amortize repeated evaluations across merge steps.

4.4.2 Fast Feasibility and Cost

Discrete candidate-time search. STAGEVAL searches over a discrete time set $\mathcal{T}(C)$ rather than continuous time. We construct $\mathcal{T}(C)$ by unioning profiled runtimes of all candidate configurations for modules in C , sort the set, and binary-search the smallest feasible time T . This bounds evaluation complexity by $O(\log |\mathcal{T}(C)|)$ feasibility checks and ties resolution to profiling granularity.

Heuristic feasibility with packing constraints. Given a target time T , STAGEVAL selects for each module a configuration whose predicted runtime is $\leq T$ and then checks whether the resulting replicas can be packed onto GPUs. We use a First-Fit Decreasing (FFD) packing heuristic: replicas are sorted by decreasing quota, then placed onto GPUs while enforcing (i) per-GPU SM budget and (ii) replica anti-affinity. If packing succeeds, T is feasible; otherwise infeasible. STAGEVAL returns the minimal feasible T and the corresponding assignment plan $A(C)$.

Algorithm 1 Profile-Guided Greedy Agglomeration for One Stage

Require: Stage module set M , profiles Q , GPU set U
Ensure: Stage partition P and assignment plan A

```

1:  $P \leftarrow \{\{m\} \mid m \in M\}$  ▷ serial baseline
2: for all  $C \in P$  do
3:    $(T(C), A(C)) \leftarrow \text{STAGEVAL}(C, Q, U)$ 
4: while  $|P| > 1$  do
5:    $\Delta_{\text{best}} \leftarrow 0; \text{Best} \leftarrow \perp$ 
6:   for  $(C_a, C_b) \in \text{Pairs}(P)$  do
7:      $C \leftarrow C_a \cup C_b$ 
8:      $(T(C), A(C)) \leftarrow \text{STAGEVAL}(C, Q, U)$  ▷ cached if seen
9:      $\Delta \leftarrow (T(C_a) + T(C_b)) - T(C)$ 
10:    if  $\Delta > \Delta_{\text{best}}$  then
11:       $\Delta_{\text{best}} \leftarrow \Delta; \text{Best} \leftarrow (C_a, C_b, C)$ 
12:    if  $\Delta_{\text{best}} \leq \epsilon$  then
13:      break
14:     $(C_a, C_b, C) \leftarrow \text{Best}$ 
15:     $P \leftarrow (P \setminus \{C_a, C_b\}) \cup \{C\}$ 
16:    Update  $A$  with  $A(C)$ 
17: return  $P, A$ 

```

Refinement. After greedy agglomeration converges, we optionally refine the final group-to-GPU binding using a CP-SAT solver (OR-Tools) to tighten feasibility and reduce packing suboptimality. If refinement fails within the time budget, we fall back to the heuristic binding.

4.4.3 Complexity and Quality

The greedy loop considers $O(r^2)$ candidate merges when r groups remain and reduces r monotonically from n to 1, yielding $O(n^3)$ merge evaluations in the worst case. Each evaluation triggers STAGEVAL, which performs $O(\log |\mathcal{T}(C)|)$ packing checks. Caching substantially reduces repeated work in practice. Overall, the design replaces the intractable B_n partition search with a bounded, monotone merge procedure, while still exploring high-gain colocations guided by profiled scaling and overhead models.

5 Implementation

Apollo is implemented as a complete end-to-end system that integrates profiling, scheduling, and execution for heterogeneous, colocation-aware LMM training. The implementation consists of three modules: *ApolloProfiler*, *ApolloScheduler*, and *ApolloEngine*. *ApolloProfiler* contains about 3K LoC, *ApolloScheduler* about 2K LoC, and *ApolloEngine* about 7K LoC. The system is engineered around two versioned artifacts: a profiling output (ProfileBundle) consumed by the scheduler, and an executable scheduling output (Plan) enforced by the runtime.

ApolloProfiler. ApolloProfiler runs an offline profiling harness under NVIDIA MPS [40] to sweep compute quotas and record each module’s scaling curve, storing the resulting samples and fitted surfaces in ProfileBundle.

ApolloScheduler. ApolloScheduler exposes a single planning interface: `Schedule(ProfileBundleList) → Plan`. It searches for a colocation-aware schedule using `hasp_search()` and resolves feasibility and detailed resource assignment via `solve_assignment()`.

ApolloEngine. ApolloEngine is a PyTorch-based distributed training runtime that executes Plan. It constructs communication groups via `torch.distributed.new_group()`. Sub-GPU enforcement is implemented Green Context Manager built on `sgl-kernel` [52, 66], which provides full-lifecycle management of quota-specific streams.

6 Evaluation

6.1 Experimental Setup

Experimental Setup. All experiments are conducted on a GPU cluster consisting of $4 \times$ NVIDIA A800 nodes. Each node is equipped with 64 CPU cores, 500 GB host memory, and $8 \times$ NVIDIA A800 GPUs. GPUs within the same node are interconnected via NVLink. Inter-node communication is supported by $4 \times$ 200 Gbps InfiniBand NICs per node.

Baselines. We compare *Apollo* against representative baselines spanning three classes: (i) job-isolated schedulers that optimize each training job independently, *DistMM*; (ii) job-coordinated schedulers that coordinate execution order across jobs, *Spindle*; and (iii) widely used parallel baselines, including Data Parallel training (DDP) and NVIDIA MPS.

Workloads. We evaluate *Apollo* using representative multi-LMM jobs workloads, including Qwen3-VL, Unified-IO 2, ImageBind, and OFASys. We run real experiments on a 2-node (16-GPU) cluster and a 4-node (32-GPU) cluster, and conduct simulation studies for larger-scale settings with 32 nodes and 64 nodes.

Metrics. Our primary metric is the makespan of a concurrent job batch, defined as the time from the earliest job start to the latest job completion. To quantify GPU-side utilization, we measure actual compute utilization using the average number of compute warps in flight per GPU. We further evaluate (i) decision quality via the prediction error of colocation overhead and (ii) system overhead using per-iteration time breakdown and the scheduler plan-search latency.

6.2 End-to-End Analysis

End-to-End Comparison against Baselines. Figure 9 compares the makespan of each job batch under different numbers of concurrent training jobs. Compared to the SOTA scheduler *Spindle*, *Apollo* achieves $1.14\times$ – $1.33\times$ speedup on the 32-GPU cluster. The largest gaps appear on the most saturated Qwen3-VL and Unified-IO 2 settings on 32 GPUs, where *Apollo* reaches up to $1.49\times$ over *DistMM* and up to $1.71\times$ over NVIDIA MPS. On 16 GPUs, *Apollo* also improves makespan by up to $1.55\times$ over MPS on Unified-IO 2.

DDP and MPS represent mainstream parallel execution baselines: DDP enforces rigid GPU ownership, while MPS enables sharing without scheduler-driven quota decisions and remains sensitive to interference. *DistMM* improves intra-job mapping but is constrained by job-level boundaries, and *Spindle* coordinates competing jobs but does not optimize sub-GPU spatial packing across heterogeneous modules. These constraints explain the consistent end-to-end advantage of *Apollo* across all four workloads in Figure 9.

GPU Computation Saturation. We use *Compute Warps in Flight* to capture SM-level concurrency, since effective spatial packing

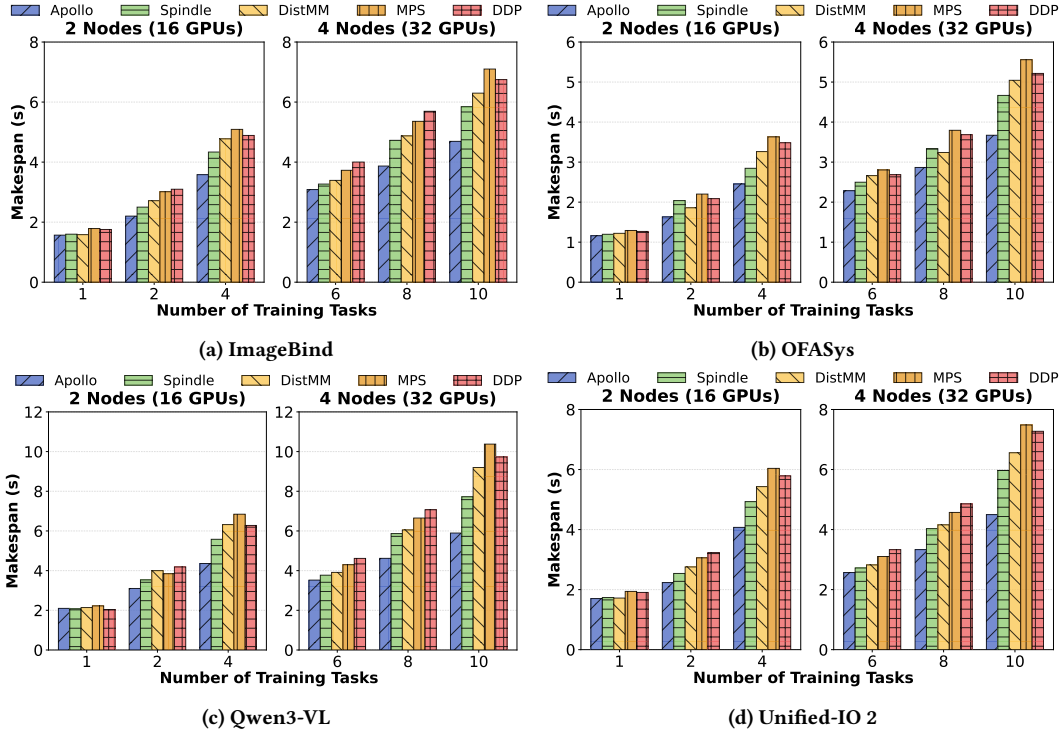


Figure 9: End-to-End Evaluation: Comparing Makespan across LMM jobs (ImageBind, OFASys, Qwen3-VL, and Unified-IO 2).

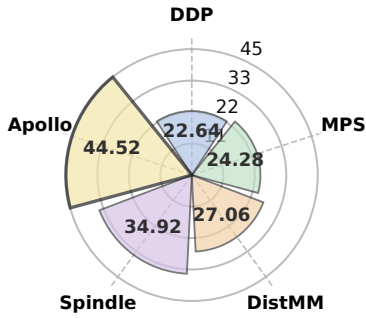


Figure 10: GPU warps-in-flight comparison across systems (10 jobs, 32 GPUs).

should keep more active work resident on SMs and reduce compute bubbles that arise from heterogeneous modules and rigid execution boundaries.

Figure 10 reports this metric for the 10-job batch selected from ImageBind jobs on 32 GPUs. *Apollo* achieves the highest saturation at 44.52% warps in flight, which is a 27.5% increase over the SOTA baseline *Spindle* (34.92%). This gap provides direct evidence that *Apollo*'s spatial multiplexing keeps substantially more useful work resident on SMs, aligning with its consistent end-to-end makespan improvements in Figure 9.

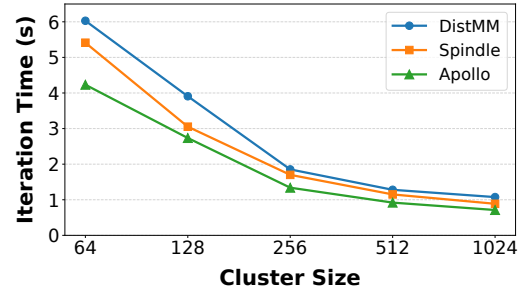


Figure 11: Simulated end-to-end time as cluster scales from 64 to 1024.

6.3 Microscopic Performance Deep Dive

6.3.1 Larger Cluster Scale Simulation

To study cluster scales beyond our physical testbed, we use a profile-driven simulator that extrapolates module runtime from measured scaling curves. For module type m under SM quota s , we start from an inverse-DP baseline and add a small fitted correction:

$$T(m; DP, s) \approx \frac{T(m; 1, s)}{DP} + r(DP, s), \quad (18)$$

where $r(DP, s)$ is a lightweight linear residual in $\log_2(DP)$ and $(DP - 1)/DP$.

Validation on measured 32-GPU profiles. We validate the simulator on ImageBind using our measured 32-GPU profiles by

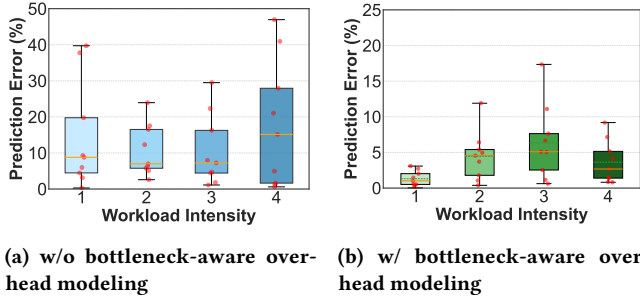


Figure 12: Prediction error comparison with and without bottleneck-aware colocated overhead modeling.

holding out DP=32 and predicting its runtime from lower-DP samples. The simulator achieves MAPE=8.35% and median absolute percentage error Median APE=5.69% at DP=32, indicating that the fitted residual captures the main deviation beyond the inverse-DP baseline and is sufficiently accurate for plan-level what-if analysis.

Scaling study via extrapolation to large cluster scales. To present the scaling results independent of absolute time, we report speedup over the baselines. From Figure 11, Apollo consistently outperforms both DistMM and Spindle across all simulated scales. The advantage persists and slightly increases at larger scales: at 1024 GPUs, Apollo reaches 1.51 $\times$ speedup over DistMM and 1.25 $\times$ over Spindle. These speedups indicate that the benefit of quota-aware colocation remains robust as cluster scale grows.

6.3.2 Performance Model Prediction Accuracy

To validate the reliability of our scheduling decisions, we compare the accuracy of *Apollo*'s performance model against a Strawman baseline that ignores resource contention. As shown in Figure 12, the Strawman approach fails to account for memory bandwidth saturation, leading to relative errors that escalate rapidly to nearly 40% as job concurrency increases. In contrast, *Apollo*'s model maintains a median absolute prediction error consistently below 5% by explicitly incorporating contention features into its regression analysis, thereby effectively capturing the non-linear interference penalties inherent in spatial multiplexing.

Although minor outliers with higher prediction errors are observed in Figure 12b, these specifically originate from extremely skewed compute allocations (1:9 SM split) that induce volatile interference patterns. However, this degradation has a negligible impact on practical decision-making. Since such imbalanced configurations inherently result in suboptimal execution efficiency, they are naturally filtered out by the HASP algorithm during the search for minimum global makespan. Consequently, the scheduler primarily operates within balanced allocation regions, where our model preserves high fidelity to guide optimal orchestration.

6.3.3 Efficiency of HASP Searching

We further evaluate the computational scalability of the scheduling algorithm itself, which is critical for online decision-making. Figure 13 compares the search latency of *Apollo*'s optimized strategy against a Brute Force approach and a Basic (non-hierarchical) implementation across varying job counts. The Brute Force method

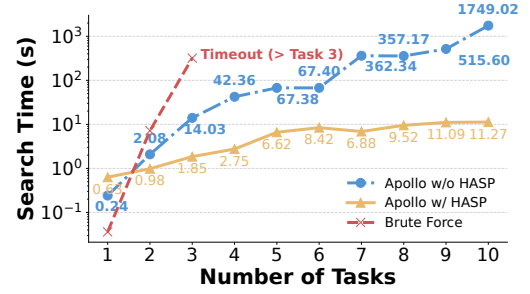


Figure 13: Allocation search time versus the number of concurrent jobs.

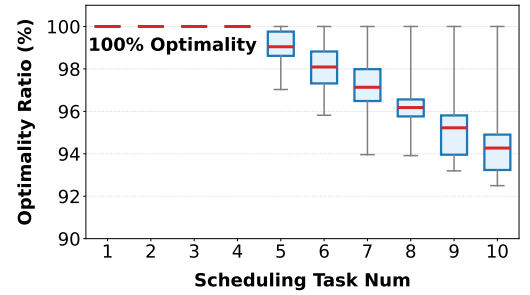


Figure 14: Optimality ratio of HASP-generated schedules across different numbers of concurrent tasks.

proves intractable, hitting a timeout threshold (marked at > 300s) with as few as 3 jobs due to the exponential explosion of the combination space. The Basic *Apollo* implementation, while better, still exhibits rapid latency growth, reaching approximately 1749 seconds for 10 jobs, which is prohibitively slow for dynamic rescheduling.

Apollo strategy which leverages the HASP algorithm, demonstrates near-linear scalability, maintaining a search time of only 11.27 seconds even with 10 concurrent jobs⁴. This significant reduction in decision latency, representing a speedup of several orders of magnitude compared to the Basic approach, ensures that *Apollo* can generate optimal colocation plans in real-time, thereby allowing the cluster to adapt swiftly to dynamic workload changes.

We assess the decision quality of our scheduling algorithm by comparing the orchestration plans generated by HASP against the theoretical global optimum derived from an exhaustive Brute Force solver. As illustrated in Figure 14, HASP consistently achieves 100% optimality for workloads with up to 4 concurrent jobs on the 32-GPU cluster, effectively identifying the best spatial partition without error. Even as the combinatorial complexity scales with more than 4 jobs, HASP maintains a near-optimal performance, with the optimality ratio sustaining above 94.27%. When combined with the significant reduction in decision latency as demonstrated in Figure 13, this result confirms that *Apollo* strikes a critical balance between orchestration optimality and search efficiency, ensuring high-quality scheduling for real-time large-scale training.

6.3.4 Efficiency of Green Context Controller.

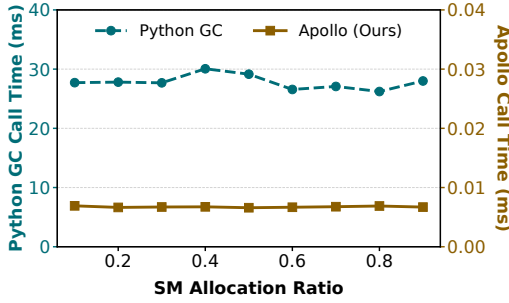


Figure 15: Green context controller overhead.

We evaluate the efficiency of Apollo’s lifecycle management against a baseline utilizing dynamic creation and Python garbage collection. As shown in Figure 15, the baseline incurs a substantial and fluctuating latency of 26 ~ 30 ms per iteration due to repetitive initialization and non-deterministic GC pauses, making it unsuitable for high-frequency training loops.

Apollo pre-allocation strategy reduces this overhead to a negligible 0.007 ms. Crucially, this near-zero cost remains consistent across varying compute allocation cases, representing a speedup of over three orders of magnitude compared to the baseline. This confirms that our mechanism effectively decouples resource management from execution scale, enabling fine-grained spatial multiplexing with virtually zero impact on training throughput.

6.4 Overhead Analysis

Overhead of the *ApolloEngine*. We quantify the runtime overhead introduced by Apollo’s orchestration mechanisms. Figure 16 decomposes the average iteration time into three components: actual computation (Fwd & Bwd), communication synchronization (*ApolloComm*), and environment initialization (*ApolloInit*). As the number of concurrent jobs increases from 6 to 10, the coordination overhead exhibits a modest upward trend due to the increased complexity of managing a denser device mesh.

Specifically, in the 10-job scenario, the overhead for Green Context initialization and lifecycle management (*ApolloInit*) is restricted to 5.34%, while the overhead for cross-group communication and synchronization (*ApolloComm*) accounts for 10.36%. This quantitative breakdown indicates that even under high concurrency, nearly 85% of the iteration time is dedicated to effective computation. The initialization cost stabilizes around 5%, proving that Apollo’s runtime management is lightweight enough to support fine-grained spatial sharing without becoming a performance bottleneck.

7 Discussions

Kernel Level Allocation and Scheduling. To address the computational complexity in multi-LMM training jobs, PipeWeaver [61] introduces a fine-grained scheduling mechanism rooted in kernel-level characterization [22]. The system conducts deep profiling of computational kernels to construct a high-fidelity execution prediction model [29, 69], which accurately captures performance volatility under dynamic workloads. Leveraging these predictive insights, PipeWeaver enforces dynamic orchestration and resource allocation

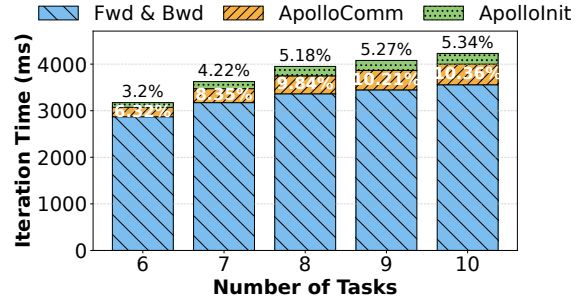


Figure 16: Job execution time breakdown.

at the kernel granularity. This approach achieves a precise alignment between computational demands and hardware resources, effectively mitigating the structural load imbalances caused by data dynamicity in multi-LMM job training.

Heterogeneous Cluster Scheduling. Addressing the disparities in GPU capabilities and interconnect bandwidths necessitates a heterogeneity-aware scheduling mechanism [34, 54] that moves beyond homogeneous assumptions. By explicitly modeling hardware performance gaps, the system must intelligently map compute-intensive modules to high-performance accelerators while optimizing data placement to mitigate communication bottlenecks. This introduces a complex search problem for parallel configurations [49], requiring advanced algorithms to harmonize diverse hardware characteristics with the heterogeneous resource demands of LMM training.

RLHF Training Scenario. Alignment paradigms such as PPO [51], GRPO [53], and ReMax [26] introduce orchestration complexity by coupling distinct roles, including Actor, Critic, and Reward models, within a unified pipeline. These components exhibit disparate computational profiles that render static resource partitioning inefficient. Apollo’s orchestration strategy generalizes effectively to this domain by treating these roles as heterogeneous modules. By dynamically optimizing resource distribution across workers, Apollo can identify the optimal allocation scheme to maximize the throughput of multi-role RLHF training.

8 Conclusion

We presented *Apollo*, a distributed training framework that exploits spatial multiplexing to mitigate module-level heterogeneity in multi-LMM jobs training. *Apollo* integrates (i) a Lifecycle-Aware Green Context Controller for low-overhead context management, (ii) a Bottleneck-Aware performance model that predicts colocation interference, and (iii) the HASP scheduling algorithm that efficiently searches quota-aware colocation plans to minimize the makespan of job batches. Experiments on a 32-GPU cluster show that Apollo delivers $1.14\times$ – $1.33\times$ end-to-end speedup over Spindle. Simulations further confirm that Apollo’s benefit remains robust as cluster scale grows, reaching up to $1.25\times$ speedup over Spindle at 1024 GPUs. Apollo demonstrates that quota-aware spatial multiplexing is a practical and effective direction for high-efficiency LMM training in shared GPU clusters.

References

- [1] The doubao visual language model officially released, with general model capability fully comparable to gpt-4o. ByteDance Seed Blog, Dec. 2024.
- [2] Longcat-image blog. Meituan, Dec. 2025.
- [3] Longcat-image: Open-source image generation and editing models. GitHub repository, Meituan LongCat Team, 2025.
- [4] ACHIAM, J., ADLER, S., AGARWAL, S., AHMAD, L., AKKAYA, I., ALEMAN, F. L., ALMEIDA, D., ALTENSCHMIDT, J., ALTMAN, S., ANADKAT, S., ET AL. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [5] ANTHROPIC. Introducing claude 4. Company announcement, 2025.
- [6] APACHE AIRFLOW. Dags. Apache Airflow Documentation, 2025. Accessed 2026-01-20.
- [7] BAI, J., ET AL. Ofasys: A multi-modal multi-task learning system for building generalist models. *arXiv*, 2022. arXiv:2212.04408.
- [8] BAI, S., CAI, Y., CHEN, R., ET AL. Qwen3-vl technical report. *arXiv*, 2025. arXiv:2511.21631.
- [9] BYTEDANCE SEED TEAM. Doubao releases two video generative models: Multiple vertical models are newly launched. <https://seed.bytedance.com/en/blog/doubao-releases-two-video-generative-models-multiple-vertical-models-are-newly-launched-and-the-research-and-development-of-basic-models-is-in-full-swing>, 2024. Accessed: 2026-02-04.
- [10] CAO, J., GUAN, Y., QIAN, K., GAO, J., XIAO, W., DONG, J., FU, B., CAI, D., AND ZHAI, E. Crux: Gpu-efficient communication scheduling for deep learning training. In *ACM SIGCOMM '24* (2024).
- [11] CHEN, Z., WANG, W., CAO, Y., ET AL. Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. *arXiv*, 2024. arXiv:2412.05271.
- [12] CHOI, S., LEE, S., KIM, Y., PARK, J., KWON, Y., AND HUH, J. Serving heterogeneous machine learning models on {Multi-GPU} servers with {Spatio-Temporal} sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)* (2022), pp. 199–216.
- [13] FENG, W., CHEN, Y., WANG, S., PENG, Y., LIN, H., AND YU, M. Optimus: Accelerating {Large-Scale} {Multi-Modal} {LLM} training by bubble exploitation. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)* (2025), pp. 161–177.
- [14] GEMINI TEAM, GOOGLE. Gemini: A family of highly capable multimodal models. *arXiv*, 2023. arXiv:2312.11805; Accessed 2026-01-20.
- [15] GEMINI TEAM, GOOGLE. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. Technical report, 2025.
- [16] GIRDHAR, R., EL-NOUBY, A., LIU, Z., SINGH, M., ALWALA, K. V., JOULIN, A., AND MISRA, I. Imagebind: One embedding space to bind them all. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2023).
- [17] GOOGLE. Introducing gemini: our largest and most capable AI model. Google Blog, Dec. 2023. Accessed 2026-01-20.
- [18] GU, D., AND ET AL. Elasticflow: An elastic serverless training platform for distributed deep learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (2023), ACM.
- [19] GU, J., CHOWDHURY, M., SHIN, K. G., ZHU, Y., JEON, M., QIAN, J., LIU, H., AND GUO, C. Tiesias: A GPU cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)* (Boston, MA, Feb. 2019), USENIX Association, pp. 485–500.
- [20] HETLAND, C., TZIANTZIOULIS, G., SUCHY, B., LEONARD, M., HAN, J., ALBERS, J., HARDAVELLAS, N., AND DINDA, P. A. Paths to fast barrier synchronization on the node. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '19)* (2019), pp. 109–120.
- [21] HUANG, J., ZHANG, Z., ZHENG, S., QIN, F., AND WANG, Y. {DISTMM}: Accelerating distributed multimodal model training. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)* (2024), pp. 1157–1171.
- [22] HUANG, S., ET AL. Neutrino: Fine-grained gpu kernel profiling via assembly-layer probing. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)* (2025), USENIX Association.
- [23] HUANG, Y., CHENG, Y., BAPNA, A., FIRAT, O., CHEN, M., CHEN, D., LEE, H., NGIAM, J., LE, Q. V., WU, Y., AND CHEN, Z. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)* (2019). arXiv:1811.06965.
- [24] KUBEFLOW. Pipeline. Kubeflow Pipelines Documentation, June 2025. Accessed 2026-01-20.
- [25] LI, H., ET AL. Interference-aware opportunistic job placement for shared gpu clusters. *Future Generation Computer Systems* (2024). ScienceDirect entry.
- [26] LI, Z., ET AL. Remax: A simple, effective, and efficient reinforcement learning method for aligning large language models. *arXiv*:2310.10505, 2023.
- [27] LIN, Z., XU, H., CHEN, G., ZHANG, X., AND LU, Y. Bullet: Boosting gpu utilization for llm serving via dynamic spatial-temporal orchestration. *arXiv*:2504.19516, 2025.
- [28] LIU, H., LI, C., WU, Q., AND LEE, Y. J. Visual instruction tuning. *Advances in neural information processing systems* 36 (2023), 34892–34916.
- [29] LU, G., ET AL. Distsim: A performance model of large-scale hybrid distributed dnn training. *arXiv*:2306.08423, 2023.
- [30] LU, J., CLARK, C., LEE, S., ZHANG, Z., KHOSLA, S., MARTEN, R., HOIEM, D., AND KEMBHAVI, A. Unified-io 2: Scaling autoregressive multimodal models with vision, language, audio, and action. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024).
- [31] MAHAJAN, K., BALASUBRAMANIAN, A., SINGHVI, A., VENKATARAMAN, S., AKELLA, A., PHANISHAYEE, A., AND CHAWLA, S. Themis: Fair and efficient GPU cluster scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)* (Santa Clara, CA, Feb. 2020), USENIX Association, pp. 289–304.
- [32] MAO, H., SCHWARZKOPF, M., VENKATKRISHNAN, S. B., MENG, Z., AND ALIZADEH, M. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM '19)* (2019), pp. 270–288.
- [33] NARAYANAN, D., HARLAP, A., PHANISHAYEE, A., SESHADRI, V., DEVANUR, N. R., GANGER, G. R., GIBBONS, P. B., AND ZAHARIA, M. Pipedream: Generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)* (2019).
- [34] NARAYANAN, D., SANTHANAM, K., KAZHAMIKA, F., PHANISHAYEE, A., AND ZAHARIA, M. Heterogeneity-Aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)* (Nov. 2020), USENIX Association, pp. 481–498.
- [35] NVIDIA. CUDA C Programming Guide: Green Contexts. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/green-contexts.html>, 2025. Accessed 2026-01-28.
- [36] NVIDIA. CUDA Driver API: Green Contexts. https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__GREEN__CONTEXTS.html, 2025. Accessed 2026-01-28.
- [37] NVIDIA. Multi-instance gpu user guide. NVIDIA Documentation, 2025. Accessed 2026-01-20.
- [38] NVIDIA CORPORATION. Cuda c++ programming guide: Green contexts. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/green-contexts.html>, 2025. Accessed: 2026-02-04.
- [39] NVIDIA CORPORATION. Cuda multi-process service: Architecture. <https://docs.nvidia.com/deploy/mps/architecture.html>, 2025. Accessed: 2026-02-04.
- [40] NVIDIA CORPORATION. Cuda multi-process service (mps) overview. https://docs.nvidia.com/deploy/pdf/CUDA_Multi_Process_Service_Overview.pdf, 2025. Accessed: 2026-02-04.
- [41] NVIDIA CORPORATION. CUDA Programming Guide: Programming model (streaming multiprocessors). <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/concepts.html>, 2025. Accessed: 2026-02-04.
- [42] NVIDIA CORPORATION. Cuda runtime api: Stream management (priority semantics). https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__STREAM.html, 2025. Accessed: 2026-02-04.
- [43] NVIDIA CORPORATION. CUDA Toolkit Documentation. <https://docs.nvidia.com/cuda/>, 2026. Accessed: 2026-02-05.
- [44] OPENREVIEW SUBMISSION. Pipetune: Tuning pipeline parallelism for efficient vision-language model training. OpenReview, ICLR 2026 submission, 2026. Submitted: 2025-09-19; last modified: 2026-01-26; Accessed: 2026-02-04.
- [45] PASZKE, A., GROSS, S., MASSA, F., LERER, A., BRADBURY, J., CHANAN, G., KILLEEN, T., LIN, Z., GIMELSHEIN, N., ANTIGA, L., ET AL. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [46] PENG, Y., BAO, Y., CHEN, Y., AND WU, C. Optimus: An efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth EuroSys Conference* (2018), ACM.
- [47] PYTHON SOFTWARE FOUNDATION. gc — garbage collector interface. <https://docs.python.org/3/library/gc.html>, 2026. Accessed: 2026-02-04.
- [48] PYTORCH CONTRIBUTORS. Getting started with distributed data parallel (ddp). https://docs.pytorch.org/tutorials/intermediate/ddp_tutorial.html, 2019. Accessed: 2026-02-04.
- [49] QIAO, A., ET AL. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)* (2021), USENIX Association.
- [50] RADFORD, A., KIM, J. W., HALLACY, C., RAMESH, A., GOH, G., AGARWAL, S., SASTRY, G., ASKELL, A., MISHKIN, P., CLARK, J., KRUEGER, G., AND SUTSKEVER, I. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning* (2021), M. Meila and T. Zhang, Eds., vol. 139 of *Proceedings of Machine Learning Research*, PMLR, pp. 8748–8763.
- [51] SCHULMAN, J., WOLSKI, F., DHARIWAL, P., RADFORD, A., AND KLIMOV, O. Proximal policy optimization algorithms. *arXiv*:1707.06347, 2017.
- [52] SGL-PROJECT. sgl-kernel (sglang kernel library). <https://github.com/sgl-project/sglang/tree/main/sgl-kernel>, 2026. Accessed: 2026-02-04.
- [53] SHAO, Z., WANG, P., ZHU, Q., ZHANG, R., QIAN, J., WEI, Y., CHEN, Y., YAN, Y., LIU, Y., WANG, Z., HU, Y., LI, M., JIAO, W., HUANG, Z., SHI, S., ET AL. Deepseek-math: Pushing the limits of mathematical reasoning in open language models.

- arXiv:2402.03300, 2024. Introduces Group Relative Policy Optimization (GRPO).
- [54] SUBRAMANYA, S. J., ARFEEN, D., LIN, S., QIAO, A., JIA, Z., AND GANGER, G. R. Sia: Heterogeneity-aware, goodput-optimized ml-cluster scheduling. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)* (2023).
- [55] SURESH, L., CANINI, M., SCHMID, S., AND FELDMANN, A. Building scalable and flexible cluster managers using declarative programming. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)* (2020), USENIX Association.
- [56] TSCHANNEN, M., ET AL. Siglip 2: Multilingual vision-language encoders with improved semantic understanding. arXiv, 2025. arXiv:2502.14786.
- [57] UM, T., OH, B., SEO, B., KWEUN, M., KIM, G., AND LEE, W.-Y. Fastflow: Accelerating deep learning model training with smart offloading of input data pipeline. *Proceedings of the VLDB Endowment (PVLDB)* 16, 5 (2023), 1086–1099.
- [58] VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, L., AND POLOSUKHIN, I. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [59] WANG, J., WANG, Y., HAN, M., AND CHEN, R. Colocating ml inference and training with fast gpu memory handover. In *USENIX Annual Technical Conference (ATC)* (2025), USENIX Association.
- [60] WANG, Y., ZHU, S., FU, F., MIAO, X., ZHANG, J., ZHU, J., HONG, F., LI, Y., AND CUI, B. Spindle: Efficient distributed training of multi-task large models via wavefront scheduling. arXiv, 2024. arXiv:2409.03365.
- [61] XUE, Z., HU, H., CHEN, X., JIANG, Y., SONG, Y., MI, Z., ZHU, Y., JIANG, D., XIA, Y., AND CHEN, H. Pipeweaver: Addressing data dynamicity in large multimodal model training with dynamic interleaved pipeline. arXiv, 2025. arXiv:2504.14145.
- [62] YANG, Y., MERLINA, A., SONG, W., YUAN, T., BIRMAN, K., AND VITENBERG, R. Navigator: A decentralized scheduler for latency-sensitive ml workflows. arXiv:2402.17652, 2024.
- [63] YE, Z., GAO, W., HU, Q., SUN, P., WANG, X., LUO, Y., ZHANG, T., AND WEN, Y. Deep learning workload scheduling in gpu datacenters: A survey. *ACM Computing Surveys* 56, 6 (2024).
- [64] ZHANG, Z., ZHENG, S., WANG, Y., CHIU, J., KARYPIS, G., CHILIMBI, T., LI, M., AND JIN, X. Mics: Near-linear scaling for training gigantic model on public cloud. *Proceedings of the VLDB Endowment (PVLDB)* 16, 1 (2022), 37–50.
- [65] ZHANG, Z., ZHONG, Y., JIANG, Y., HU, H., SUN, J., GE, Z., ZHU, Y., JIANG, D., AND JIN, X. Disttrain: Addressing model and data heterogeneity with disaggregated training for multimodal large language models. In *Proceedings of the ACM SIGCOMM 2025 Conference* (2025), pp. 24–38.
- [66] ZHENG, L., YIN, L., XIE, Z., SUN, C., HUANG, J., YU, C. H., CAO, S., KOZYRAKIS, C., STOICA, I., GONZALEZ, J. E., BARRETT, C., AND SHENG, Y. Sglang: Efficient execution of structured language model programs. arXiv:2312.07104, 2024. v2 posted 2024-06-06; Accessed: 2026-02-04.
- [67] ZHENG, P., PAN, R., KHAN, T., VENKATARAMAN, S., AND AKELLA, A. Shockwave: Fair and efficient cluster scheduling for dynamic adaptation in machine learning. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)* (2023), USENIX Association.
- [68] ZHENG, S., CHEN, X., SHI, Z., YAN, L.-Q., AND XU, K. Gpu coroutines for flexible splitting and scheduling of rendering tasks. *ACM Transactions on Graphics* 43, 6 (2024).
- [69] ZHU, H., ET AL. Accurately estimating the efficacy of optimizations for dnn training. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)* (2020), USENIX Association.